CSC490 Assignment 3

Team Members
Minh Le (1007894432), Prashanth Shyamala (1008819021),
Shaan Purewal (1008332939), York Ng (1009153577)

1 Part One: Architecture Review

1.1 nanochat Architecture

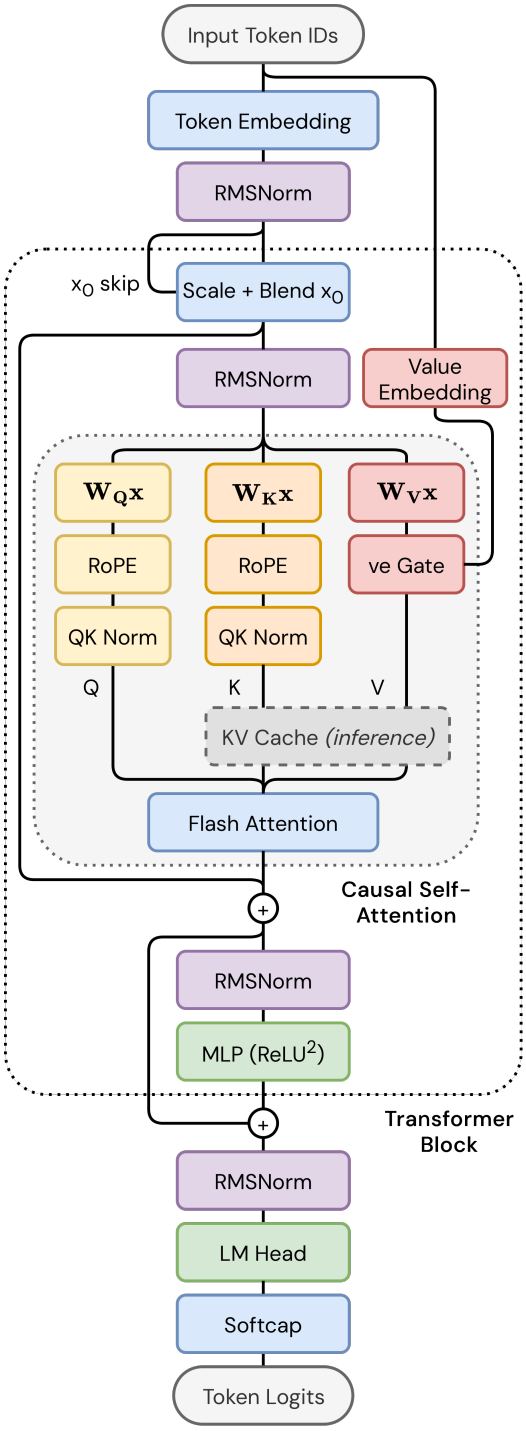

Figure 1: Architecture of the nanochat LLM

nanochat is a decoder-only transformer language model, and its architecture is depicted in Figure 1. As of its February 2026 update, it departs from a vanilla transformer in the following ways:

Embeddings & output

- Untied input/output embeddings (`wte` and `lm_head` are separate parameters with different initializations and learning rates)
- RMSNorm applied immediately after the token embedding, before any blocks
- Logit softcapping ($15 \cdot \tanh(\text{logits}/15)$) bounds outputs to [-15, +15], computed in float32

Residual stream

- Pre-norm architecture: RMSNorm before each attention and MLP sub-block, implemented as a purely functional `F.rms_norm` call
- Per-layer learnable scalar pair: $x = \lambda_{\text{resid}} \cdot x + \lambda_{x_0} \cdot x_0$, where x_0 is the initial normalized embedding saved before the block loop and reinjected at every layer

Attention

- Grouped query attention (fewer K/V heads than Q heads, shared across query groups)
- Rotary positional embeddings (RoPE) applied to Q and K only, with base $\theta = 10{,}000$, precomputed and cached
- QK normalization: `F.rms_norm` applied per head to Q and K after RoPE, stabilizing attention without softcapping
- Value embeddings: on alternating layers, a separate per-layer vocabulary lookup table injects token identity directly into V via a learned gate ($v = v + 2 \cdot \sigma(W_{\text{gate}} \cdot x[\ldots, :32]) \cdot ve$)
- Sliding window attention with pattern `SSSL` (three short-context layers per one full-context layer, tiled across depth; final layer always full context), implemented via Flash Attention 3

MLP

- Two-matrix feed-forward with 4× hidden expansion and no gating or bias
- ReLU^2 activation (`F.relu(x).square()`)

Training

- Hybrid Muon + AdamW optimizer: Muon handles 2D weight matrices, AdamW handles embeddings and scalars
- All hyperparameters (width, number of heads, learning rates, training horizon) are derived automatically from a single `-depth` flag

1.2 Proposed Changes

We propose three architectural changes based on open-source language modeling research. Instead of formatting it through a table, we believe it's better to present the changes as free text to accommodate formulas.

1.2.1 SwiGLU Activation

Motivation. nanochat uses a $\texttt{ReLU}^2$ activation in the feed-forward network (FFN):

$$\text{FFN}_{\text{ReLU}^2}(x) = (\text{ReLU}(xW_1))^2 \cdot W_2$$

While $\texttt{ReLU}^2$ is computationally cheap and produces sparse activations, it discards all information from negative pre-activations and introduces a hard zero boundary that can impede gradient flow. Since the introduction of Gated Linear Units (GLU) [3] and their variants [14], SwiGLU has been adopted as the FFN activation for modern open-source LLMs, such as LLaMA [15] and Qwen [17], among others.

Technical Details. SwiGLU [14] replaces the single linear projection + activation with a gated structure:

$$\text{SwiGLU}(x, W, V) = \text{SiLU}(xW) \otimes xV$$

where $\text{SiLU}(z) = z \cdot \sigma(z)$ is the Swish activation and $\otimes$ denotes element-wise multiplication. Because GLU variants require three weight matrices instead of two (an additional gate projection), the hidden dimension is reduced by a factor of $\frac{2}{3}$ to maintain approximate parameter parity. Concretely, for an embedding dimension d, the original FFN uses a $d \to 4d \to d$ structure, while SwiGLU uses $d \to \frac{8d}{3} \to d$ (with the up-projection and gate projection fused into a single $d \to \frac{16d}{3}$ linear layer that is split post-hoc).

In our implementation, we replace the nanochat MLP as follows:

```
hidden_dim = int(8 * n_embd / 3)
c_fc = Linear(n_embd, 2 * hidden_dim)   # fused gate + value
c_proj = Linear(hidden_dim, n_embd)
# Forward:
gate, val = c_fc(x).chunk(2, dim=-1)
x = F.silu(gate) * val
x = c_proj(x)
```

Potential Impact. Shazeer's experiments [14] on the T5 transformer model showed that SwiGLU and GEGLU produced the best held-out perplexities compared to other GLU variants. The gating mechanism provides an input-dependent filter that can selectively pass or suppress information, adding representational abilities. We expect modest improvements in validation loss (bits per byte) with negligible throughput impact, as the parameter count is held approximately constant by the $\frac{8}{3}$ hidden dimension scaling.

1.2.2 Learnable RMSNorm

Motivation. nanochat uses a functional RMSNorm with no learnable parameters:

$$\bar{a}_i = \frac{a_i}{\text{RMS}(\mathbf{a})}$$

The original RMSNorm paper [19] defines the normalization *with* a learnable scale parameter g_i:

$$\bar{a}_i = \frac{a_i}{\text{RMS}(\mathbf{a})} \cdot g_i, \quad \text{where} \quad \text{RMS}(\mathbf{a}) = \sqrt{\frac{1}{n}\sum_{i=1}^{n} a_i^2}$$

Every major LLM that uses RMSNorm, including LLaMA, Mistral, Qwen, and Gemma, includes the learnable γ vector. Karpathy himself even commented in the code about the decision: "Purely functional rmsnorm with no learnable params", which we believe was due to simplicity and minimizing model's parameters. Thus, this is a natural architecture change that we can explore.

Technical Details. We introduce an `RMSNorm` module with a learnable weight vector $\gamma \in \mathbb{R}^d$, initialized to ones:

```
class RMSNorm(nn.Module):
    def __init__(self, dim):
        super().__init__()
        self.weight = nn.Parameter(torch.ones(dim))
    def forward(self, x):
        return F.rms_norm(x, (self.weight.shape[0],), self.weight)
```

This module replaces all instances of the functional `norm(x)` call throughout the model: in the embedding normalization, the pre-attention and pre-MLP norms within each block, the QK norms in attention, and the final norm before the language modeling head. The learnable parameters are trained

with AdamW at the same learning rate as the unembedding head, scaled by $\propto 1/\sqrt{d_{\text{model}}}$. They are initialized to ones so that the model starts with behavior identical to the parameter-free baseline, and the RMSNorm weights are cast to `bfloat16` along with other embedding parameters for memory efficiency.

Potential Impact. The additional parameter overhead is minimal: for a model with embedding dimension d and L layers, learnable RMSNorm adds approximately $(4L+2) \cdot d$ parameters (two norms per block, one QK norm pair per attention layer, plus embedding norm and final norm). For a depth-16 model with $d = 1024$, this is roughly 66,560 parameters, which is negligible compared to hundreds of millions of total parameters. The per-channel scale provides the model with control over feature magnitudes after normalization, which is the standard formulation in literature. We hypothesize this will yield a small but consistent improvement in loss, particularly benefiting from better gradient conditioning across layers.

1.2.3 Differential Attention

Motivation. Standard softmax attention tends to distribute probability mass across all tokens in the context, including irrelevant ones. This "attention noise" wastes model capacity and can lead to hallucination and degraded performance on tasks requiring precise information retrieval. The Differential Transformer [18] addresses this by computing attention as the difference of two softmax maps, which is analogous to noise-cancelling headphones.

Technical Details. In differential attention, the query and key projections are split into two halves, producing two independent attention maps:

$$\text{DiffAttn}(X) = \left(\text{softmax}\left(\frac{Q_1 K_1^\top}{\sqrt{d}}\right) - \lambda \cdot \text{softmax}\left(\frac{Q_2 K_2^\top}{\sqrt{d}}\right)\right) V$$

where $Q = [Q_1; Q_2]$, $K = [K_1; K_2]$ are obtained by splitting the query and key projections, and λ is a learnable scalar initialized as $\lambda_{\text{init}} = 0.8 - 0.6 \cdot e^{-0.3 \cdot l}$ for layer l. A GroupNorm is applied after the subtraction to stabilize gradient flow. The key insight is that the subtraction cancels out common-mode noise, which means tokens that both maps attend to similarly are suppressed, while tokens that are differentially relevant are amplified.

Potential Impact. [18] reported that a 6.8B Diff Transformer matches the validation loss of an 11B standard Transformer, requiring only approximately 65% of the parameters or training tokens to achieve equivalent language modeling performance. The architecture also demonstrates substantial improvements in key information retrieval, hallucination mitigation, and in-context learning robustness.

2 Part Two: Ablations on a nano nanochat

Relevant files:

```
nanochat_modal.py
nanochat/nanochat/gpt_modded.py
```

2.1 Architecture Changes

We selected SwiGLU and learnable RMSNorm as the two changes to ablate, as both modifications have clear theoretical motivations and are straightforward to integrate into nanochat's existing codebase. A side benefit of this choice is that effects are isolated for our ablation study. Each change touches a single submodule, so we can attribute any observed difference in downstream performance to that change rather than confounding interactions.

2.2 Picochat Configuration

We selected depth $d = 16$. At this depth, each training run completes in under one hour on 8 H100 GPUs, allowing us to run the four models (baseline, +SwiGLU, +Learnable RMSNorm, +Both) without costing too many credits. The models are still large enough that the architecture changes have meaningful parameters to act on: the learnable RMSNorm adds $(4 \times 16 + 2) \times d$ parameters, and SwiGLU's $\frac{8}{3}$ hidden dimension scaling is non-trivial at this size. A shallower configuration such as $d = 8$ would risk being too small for the changes to manifest meaningfully, since the representational bottleneck would dominate any performance gains.

2.3 Results

All training and testing were performed using a modified version of the Modal script in CSC490 Nanochat Tutorial [13]. Table 1 summarizes the centered accuracy across all 22 benchmark tasks and the aggregate CORE metric. All three modified models improve over the baseline (CORE: 0.165), with +Learnable RMSNorm achieving the highest individual CORE score (0.176) and +Both achieving 0.173, slightly below +Learnable RMSNorm alone (0.176).

+SwiGLU (+0.011 CORE) improves most clearly on knowledge-intensive tasks. For instance, BigBench QA Wikidata increases by 3.4 percentage points and AGI Eval LSAT-AR increases by 11.4 percentage points. In LSAT-AR, we additionally see the largest gain in a single task across all ablations. This is consistent with SwiGLU's gating mechanism providing richer input-dependent filtering in the FFN, which Shazeer [14] showed yields better held-out perplexity than ungated activations. However, +SwiGLU shows mixed results within the commonsense category. Notably, performance decreases on COPA (−10.0pp) and PIQA (−1.2pp) while improving on HellaSwag, Winograd, and Winogrande. The COPA regression in particular may reflect that SwiGLU's input-dependent gating provides less benefit for short, two-alternative forced-choice tasks where the FFN has limited context to act on.

+Learnable RMSNorm (+0.015 CORE) shows the most consistent improvements across commonsense reasoning benchmarks. For instance, Winograd improves by 5.9pp, Winogrande by 4.8pp, and BigBench Language Identification by 5.1pp. This pattern is consistent with our hypothesis that per-channel learnable scale parameters improve gradient conditioning across layers, thus benefitting tasks that require fine-grained feature discrimination. Zhang and Sennrich's original RMSNorm paper [19] included this learnable γ vector as part of the standard formulation, and every major LLM using RMSNorm (LLaMA [15], Qwen [17], etc.) retains it. Our results therefore demonstrate that nanochat's omission of it may be worth revisiting.

+Both (+0.014 CORE) demonstrates that the two changes are partially additive. It improves over +SwiGLU alone (0.164) by +0.009, confirming that adding learnable RMSNorm on top of SwiGLU yields further gains. However, +Both does not match +Learnable RMSNorm alone (0.176 vs. 0.173), suggesting that SwiGLU's performance decreases on tasks such as COPA carries over to our model. The most notable individual result for this model is BoolQ (−0.195 centered), which achieves the best score across all four runs. This task requires binary reasoning and appears to benefit from the combined

Table 1: Picochat ablation results (centered accuracy). Best result per task is **bolded**.

Task	Baseline	+SwiGLU	+Learnable RMSNorm	+Both
Commonsense Reasoning				
HellaSwag (0-shot)	0.189	0.205	**0.211**	0.184
HellaSwag (10-shot)	0.184	**0.203**	0.195	0.179
PIQA	0.332	0.320	0.328	**0.356**
COPA	**0.340**	0.240	0.320	0.320
OpenBookQA	0.112	0.109	0.109	**0.141**
CommonsenseQA	0.088	**0.113**	0.103	0.048
Winograd	0.180	0.202	**0.238**	0.202
Winogrande	0.064	0.084	**0.112**	0.076
Reading Comprehension & QA				
ARC Easy	**0.453**	0.437	0.437	0.448
ARC Challenge	**0.096**	0.053	0.091	0.072
BoolQ	-0.432	-0.268	-0.274	**-0.195**
SQuAD	0.174	0.176	0.164	**0.178**
CoQA	0.188	**0.194**	0.176	0.174
LAMBADA	0.326	**0.350**	0.314	0.346
Knowledge & Reasoning				
Jeopardy	**0.022**	0.012	0.016	**0.022**
BigBench QA Wikidata	0.398	**0.432**	0.376	0.402
AGI Eval LSAT-AR	-0.011	**0.103**	0.082	0.054
Algorithmic & Formal Tasks				
BigBench Dyck Languages	0.128	0.128	**0.136**	**0.136**
BigBench CS Algorithms	0.448	0.418	0.432	**0.452**
BigBench Operators	0.157	**0.162**	0.157	0.124
BigBench Repeat Copy	**0.031**	**0.031**	**0.031**	**0.031**
BigBench Lang. ID	0.160	0.173	**0.210**	0.206
CORE Metric	0.159	0.164	**0.176**	0.173
Δ vs. Baseline	—	+0.005	**+0.017**	+0.014

effect of richer FFN representations and better-conditioned normalization. Overall, +Both remains a reasonable choice since it recovers most of the CORE gap relative to +Learnable RMSNorm while also gaining the knowledge-retrieval improvements that SwiGLU brings on tasks such as LSAT-AR and BigBench QA Wikidata.

2.4 Post-SFT Results

Table 2: Post-SFT evaluation results for all four d=16 models.

Config	ChatCORE	ARC-E	ARC-C	MMLU	GSM8K	HumanEval
Baseline	0.2744	0.4285	0.3686	0.3287	0.0576	0.1037
+SwiGLU	0.2804	0.4558	0.3771	0.3334	0.0478	0.0915
+Learnable RMSNorm	0.2820	0.4562	0.3609	0.3334	0.0500	**0.1159**
+Both	**0.2949**	**0.4870**	**0.3908**	**0.3347**	0.0470	0.1098

Table 2 shows ChatCORE and key benchmark scores after supervised fine-tuning (SFT) for all four models. The SFT results partially diverge from the CORE evaluation: +Both achieves the highest ChatCORE (0.2949) score despite not achieving the highest base CORE, while +Learnable RMSNorm

alone, which led on base CORE, ranks third after SFT. This suggests that SwiGLU's input-dependent gating provides representations that are more amenable to instruction following, even when it does not improve raw language modelling.

The one consistent exception is GSM8K, where all three modified configurations underperform the baseline (0.0576), suggesting that the mathematical reasoning capabilities captured during pretraining are disrupted by both changes. HumanEval is the only benchmark where +Learnable RMSNorm alone outperforms +Both (0.1159 vs. 0.1098), consistent with its base CORE advantage carrying over to code generation after fine-tuning.

2.5 Training Dynamics

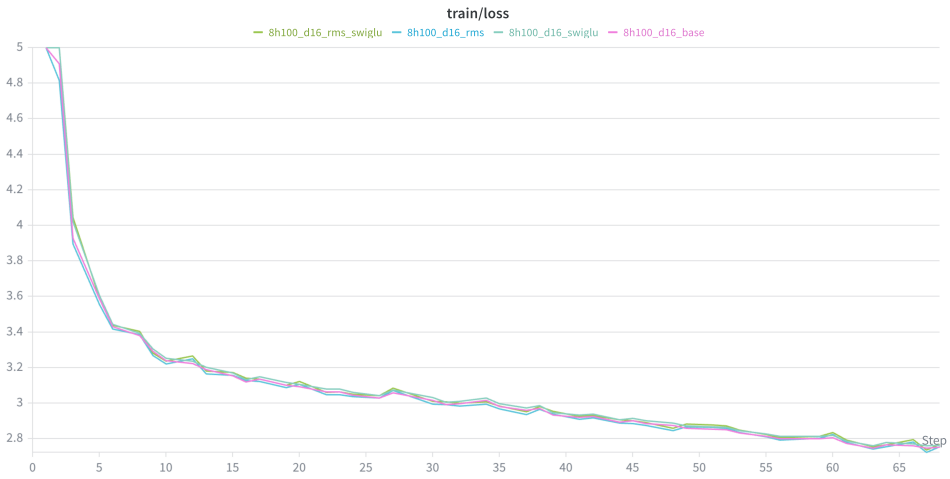

Figure 2: Training loss over number of steps for all four runs

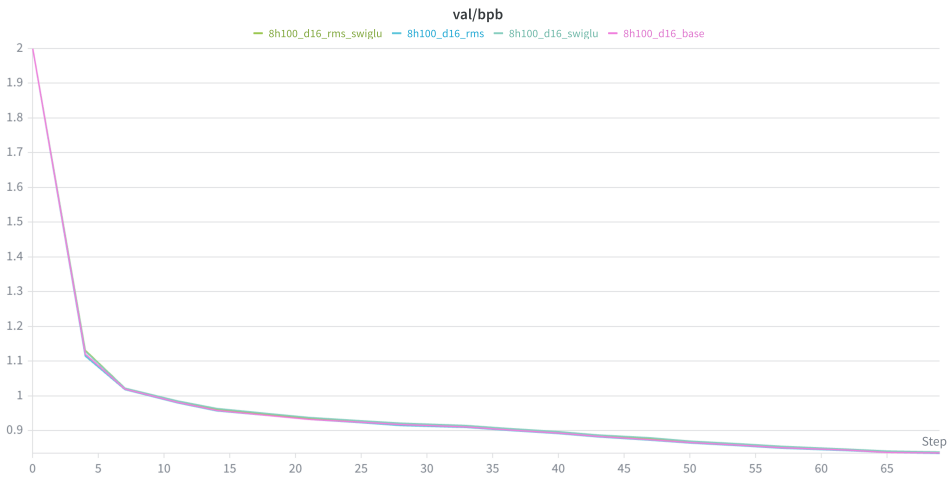

Figure 3: Validation bits-per-byte over number of steps for all four runs

Figures 2, 3, and 4 show the training loss, validation bits-per-byte, and model FLOP utilization (MFU) for all four runs respectively.

The train/loss and val/bpb curves (Figures 2 and 3) are nearly indistinguishable between all four models. All runs converge along the same trajectory, with final validation bpb values tightly clustered below 0.85. This confirms that neither SwiGLU nor learnable RMSNorm destabilizes training or causes optimization divergence, and crucially that the differences observed in downstream task performance are not artifacts of one run converging faster or slower. The tight clustering also demonstrates that at this scale, our changes do not meaningfully alter the loss landscape.

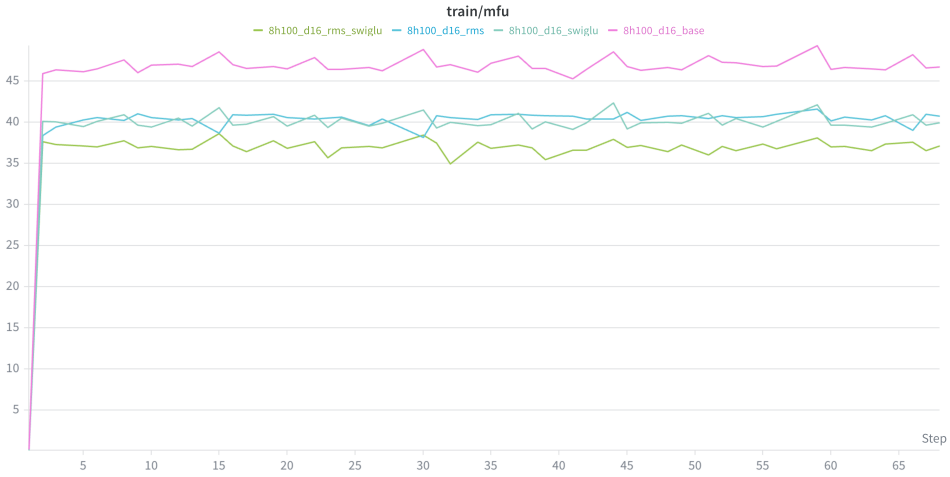

Figure 4: Model FLOPs Utilization over number of steps for all four runs

The train/mfu plot (Figure 4) tells a different story. The baseline achieves approximately 46% MFU, while +SwiGLU and +Learnable RMSNorm each settle at approximately 40%, and +Both drops further to approximately 37–38%. The reduction from SwiGLU is expected: replacing nanochat's two-matrix FFN with a three-matrix gated structure increases the number of linear operations per forward pass, even after reducing the hidden dimension by $\frac{2}{3}$ to maintain parameter parity. The small additional MFU reduction from learnable RMSNorm is consistent with the overhead of loading and applying per-channel weight vectors across all normalization sites. These MFU reductions (6pp and 9pp respectively) are modest and do not represent a practical barrier to adoption, but they are measurable and should be accounted for when estimating training costs at larger scale.

2.6 Cost of Training

Each picochat run was trained for approximately 4,700 steps on 8 H100 GPUs. The baseline completed in approximately 1,100 seconds (18 minutes), while +Both required approximately 1,400 seconds (23 minutes), representing a 27% increase in wall-clock time. This is consistent with the MFU reduction observed in Figure 4: the baseline runs at $\sim$ 46% MFU while +Both drops to $\sim$ 38%, meaning fewer useful FLOPs per second for the same hardware. A full nanochat training run using both changes would have require proportionally more GPU hours to process the same number of tokens as the baseline. Therefore, our ablation study at this smaller scale allows us to validate the changes before scaling up, allowing us to avoid spending compute on some configuration that might have shown no benefit or introduced instability.

2.7 Scalability to Larger Models

Our ablations suggest that both changes are likely to remain beneficial at larger scale. Zhang and Sennrich [19] included the learnable γ vector in their original RMSNorm formulation, and it has been retained in every major subsequent LLM that adopts RMSNorm. We will hopefully see larger performance improvements in commonsense reasoning in larger models, where there are more layers and more parameters for the per-channel scaling to condition.

SwiGLU's benefits at scale are also evident in modern LLMs, adopted by PaLM [2] (540B parameters) and LLaMA [15] (7B–65B parameters), for example. That being said, the Falcon series [1] reported no clear zero-shot uplift from SwiGLU at their model scales, suggesting the benefit is not universal and may depend on training data and model architecture. Our mixed results seen here are therefore not surprising, and the performance decrease in COPA and GSM8K could diminish further in larger models, where the gating mechanism has more capacity to leverage. The MFU cost of SwiGLU is also amortized more favorably at larger batch sizes and longer training horizons, where the additional FLOPs per step become a smaller fraction of total compute relative to memory transfer costs.

3 Part Three: Extending the context window

Relevant files:

```
nanochat/scripts/passkey_eval.py
nanochat/scripts/passkey_compare.py
nanochat_modal_part3.py
```

3.1 Experiment Design

In this section, we investigate the effect of a two-stage training schedule, where nanochat is pretrained on shorter sequences before being upscaled to a longer context. We selected 512 tokens as our reduced context window for several reasons. 512 is a natural power-of-two subdivision of 2048 ($2048/4 = 512$), which keeps batch arithmetic clean and avoids wasting GPU memory on padding. Secondly, 512 is a well-studied context length: the original BERT model [4] used 512 tokens and GPT-2 experiments ablated at this length [12]. Finally, choosing a length that is $4\times$ shorter than the target creates a large enough gap that we can clearly observe the effect of context extension in evaluation, rather than measuring noise.

Both checkpoints are trained on 128 FineWeb-EDU shards, fewer than the 240 shards recommended for the larger d24 model because the d16 model used here is substantially smaller. The first checkpoint is saved at step 1500, marking the end of the 512-token pretraining phase. Training then resumes from this checkpoint with the sequence length increased to 2048, continuing until step 2500. This provides an additional 1000 training steps at the full context length. We expect that the first phase allows the model to learn local sequence representations, while the second phase focuses on adapting attention patterns and positional dependencies to the larger context window.

3.2 Evaluation Design: Passkey Retrieval

The standard CORE benchmark used by nanochat is not ideal for evaluating long context scenarios, since most of its tasks fit comfortably within a few hundred tokens. To this end, we adopt the passkey retrieval task [11] and subsequently used as a standard needle-in-a-haystack diagnostic [9, 5]. The task embeds a random 6-digit number (the "passkey") at a controlled position within a long block of filler text and asks the model to reproduce it. This forms a information-retrieval task that requires the model to attend to the exact position where the passkey appears, regardless of the surrounding context. Because the model either outputs the correct digits or not, performance is straightforward to evaluate.

Since our checkpoints are base models without supervised fine-tuning, they are simply next-token predictors and might not be able to follow chatbot-style instructions. A prompt such as "What is the passkey? Answer with digits only." would not elicit a reliable response from our two checkpoints.

We therefore use a completion-style prompt format:

```
[filler text A]
The magic number is 123456. Please remember this number.
[filler text B]
As stated in the text above, the magic number is
```

The model's task is to continue the final sentence with the correct digits, thus does not require any instruction-following behavior. The filler text (defined in `nanochat/scripts/passkey_eval.py` consists of shuffled but stylistically consistent with the pretraining data to avoid domain mismatch.

Following Liu et al. [9], we vary the needle depth—the relative position of the passkey within the filler, from 0.0 (beginning) to 1.0 (end)—across 8 evenly spaced bins. This allows us to detect position-dependent retrieval failures, such as the "lost in the middle" phenomenon where models attend well to the beginning and end of their context but poorly to the middle. We generate 128 examples per context length, using a fixed random seed to ensure that both checkpoints are evaluated on identical prompts. Context lengths tested are 128, 256, 512, 1024, 1536, and 2048 tokens.

3.3 Results

3.3.1 Overall Accuracy

Context (tokens)	Ckpt 1 (step 1500)	Ckpt 2 (step 2500)	Δ
128	1.000	0.977	−0.023
256	1.000	0.984	−0.016
512	1.000	1.000	+0.000
1024	0.500	0.984	+0.484
1536	0.359	0.898	+0.539
2048	0.250	0.742	+0.492

Table 3: Passkey retrieval accuracy by context length. Checkpoint 1 was trained entirely at sequence length 512; Checkpoint 2 continued training at sequence length 2048. At evaluation time, both models are tested at all context lengths with RoPE embeddings extended to 2048.

From table 3, Checkpoint 1 achieves perfect accuracy at context lengths up to 512 tokens—exactly its training window—then degrades sharply, falling to 50% at 1024 tokens and then 25% at 2048. Checkpoint 2, which received 1000 additional steps at the full 2048-token context, recovers: it scores 98.4% at 1024 tokens and 74.2% at 2048. Averaged across lengths beyond the 512-token training boundary, Checkpoint 2 scores 0.906 versus Checkpoint 1's 0.432.

An interesting phenomenon is that Checkpoint 2 shows a slight regression at very short contexts (128 and 256 tokens), scoring 97.7% and 98.4% where Checkpoint 1 achieved 100%. We believe this is catastrophic forgetting [10], where the continued training at longer sequences slightly perturbed the model's short-range representations. The effect is small but it is worth noting as evidence that context extension is not entirely without drawbacks.

3.3.2 Depth Analysis

		Needle Depth							
Context	**Checkpoint**	**0.00**	**0.14**	**0.29**	**0.43**	**0.57**	**0.71**	**0.86**	**1.00**
512	Ckpt 1 (step 1500)	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
	Ckpt 2 (step 2500)	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
1024	Ckpt 1 (step 1500)	0.00	0.00	0.00	0.00	1.00	1.00	1.00	1.00
	Ckpt 2 (step 2500)	1.00	0.88	1.00	1.00	1.00	1.00	1.00	1.00
2048	Ckpt 1 (step 1500)	0.00	0.00	0.00	0.00	0.00	0.00	1.00	1.00
	Ckpt 2 (step 2500)	0.00	0.12	0.88	1.00	0.94	1.00	1.00	1.00

Table 4: Passkey retrieval accuracy by needle depth. Depth 0.00 places the passkey at the beginning of the context; 1.00 places it at the end. Checkpoint 1 shows a hard cutoff at approximately 512 tokens from the end, while Checkpoint 2 exhibits the "lost in the middle" pattern with degradation concentrated at depth 0.00–0.14.

From table 4, the position of the passkey within the context input also affects model's performance. For Checkpoint 1 at contexts beyond 512 tokens, the model retrieves the passkey only when it falls within approximately the last 512 tokens of the input. At 1024 tokens, if the passkey is in the first half of the context, the model results in 0% accuracy, while depths 0.57–1.00 yield 100%. This is exactly what we would expect: the model's positional embeddings were only trained up to position 512, so attention scores for positions beyond that range become unreliable.

For Checkpoint 2, at 2048 tokens, it achieves perfect accuracy for later depths, but degrades at the earlier depths. This is somewhat consistent with [9], where transformer models exhibit strong performance at the end of context (recency bias) and weak at the very beginning.

4 Part Four: Training The Final Nanochat

4.1 Configuration

For our final model we select d=24 with both architectural modifications from Part 2 (SwiGLU activation and learnable RMSNorm), which we again refer to as **+Both**. This depth matches Karpathy's baseline configuration [7], which allows us to directly compare our modifications with their setup. As for the +Both configuration, this is based on our ultimate goal being a useful chat model, where ChatCORE is the more relevant metric. While +Learnable RMSNorm alone achieves the highest *base* CORE at d=16, we note that +Both achieves the highest ChatCORE after fine-tuning (0.2949 vs. 0.2744 baseline). We therefore pick the +Both configuration.

To provide a fair baseline for comparison, we train our own d=24 base model to serve as our baseline, rather than relying on Karpathy's reported numbers.

4.2 Scaling Law Prediction

Here, we consider how performance would scale with depth (i.e. parameter count, model size). From the Part 2 results, we can observe the following empirical scaling trend for the +Both configuration:

- d=16: 537M parameters, 2.47B tokens, base CORE = 0.1725
- d=24: 1.38B parameters, 7.66B tokens, base CORE = 0.2504

The 2.57× parameter increase (accompanied by a 3.1× token increase to maintain the fixed ratio) yields a CORE improvement of +0.078. This is consistent with what we'd expect under power-law scaling behaviour, where each doubling of compute yields diminishing but positive returns [6]. The MFU also improves from 37.4% to 46.9% as the larger model saturates H100 tensor cores more efficiently.

4.3 Results

Table 5 summarizes results across all four combinations of scale ($d = 16$ vs $d = 24$) and configurations (baseline vs +Both). We highlight our key findings below:

Scale dominates our additions. The gap between d=16 and d=24 is substantially larger than the gap between Base and +Both at both depths. For example, going from d=16 +Both to d=24 Base improves ChatCORE by +0.061 (0.2949 → 0.3560), while the +Both adds only +0.013 at d=24 (0.3560 → 0.3690). This is expected result under scaling laws: architectural improvements are real but dominated by the impact of larger scales [6].

+Both gains hold at scale. +Both provides consistent improvements over the d=24 baseline on CORE (+0.012), ChatCORE (+0.013), ARC-Easy (+2.6pp), and ARC-Challenge (+5.4pp). This reinforces our findings in Part 2, where +Both demonstrates partially additive gains from both architectural changes, showing that it also holds in the larger model. This also confirms our justification for choosing +Both for our final model, as it indeed gives us the best SFT performance for a chat-focused model.

Performance loss in reasoning worsens at scale. At d=16, +Both shows performance loss of −0.011 in GSM8K compared to the baseline. At d=24 this gap widens to −0.018 (0.0978 → 0.0796), as we had theorized in Section 2.7. HumanEval shows a similar pattern: at d=16 +Both is roughly neutral (+0.006), but at d=24 performance decreases by −0.018 (0.1220 → 0.1037). This represents empirical evidence that SwiGLU's gating mechanism cause confusion in the structured, step-by-step reasoning required in tasks such as mathematical and code generation tasks, and that this effect becomes more pronounced as the model gains capacity. Considering that the Falcon series [1] similarly found no universal benefit from SwiGLU, we note that this is a limitation for use cases requiring strong reasoning performance.

Comparison with Karpathy's reported numbers. Our d=24 base achieves a val bpb of 0.7533 and CORE of 0.2389. We note that this underperforms Karpathy's baseline (CORE 0.2589) [8]. We attribute

Table 5: Results across scale and configuration. Best result per row is bolded.

Task	d=16 **Base**	d=16 +**Both**	d=24 **Base**	d=24 +**Both**
Base model				
Val bpb (lower = better)	0.8359	0.8390	**0.7533**	0.7552
Commonsense reasoning				
HellaSwag (0-shot)	0.176	0.184	**0.349**	0.333
HellaSwag (10-shot)	0.177	0.179	**0.343**	0.335
PIQA	0.330	0.356	**0.429**	0.426
COPA	0.320	0.320	**0.340**	**0.340**
OpenBookQA	0.112	0.141	0.195	**0.187**
CommonsenseQA	0.027	0.048	0.045	**0.041**
Winograd	0.180	0.202	**0.326**	0.304
Winogrande	0.056	0.076	0.105	**0.140**
Reading comprehension & QA				
ARC Easy	0.453	0.448	0.548	**0.548**
ARC Challenge	0.096	0.072	0.152	**0.145**
BoolQ	-0.350	-0.195	-0.250	**-0.093**
SQuAD	0.174	0.178	0.362	**0.376**
CoQA	0.188	0.174	0.259	**0.275**
LAMBADA	0.326	0.346	0.427	**0.434**
Knowledge & reasoning				
Jeopardy	0.022	0.022	0.171	**0.188**
BigBench QA Wikidata	0.398	0.402	0.510	**0.519**
AGI Eval LSAT-AR	-0.011	0.054	0.071	**0.114**
Algorithmic & formal tasks				
BigBench Dyck Languages	0.128	0.136	0.117	**0.128**
BigBench CS Algorithms	0.448	0.452	0.386	**0.388**
BigBench Operators	0.157	0.124	0.162	**0.176**
BigBench Repeat Copy	0.031	0.031	0.031	**0.031**
BigBench Lang. ID	0.160	0.206	0.178	**0.174**
CORE Metric ↑	0.159	0.173	0.239	**0.250**
Δ vs. d=24 Base	-0.080	-0.066	—	$+0.012$
After SFT				
ChatCORE ↑	0.2744	0.2949	0.3560	**0.3690**
ARC-Easy ↑	0.4285	0.4870	0.6031	**0.6288**
ARC-Challenge ↑	0.3686	0.3908	0.4573	**0.5111**
MMLU ↑	0.3287	0.3347	0.3795	**0.3890**
GSM8K ↑	0.0576	0.0470	**0.0978**	0.0796
HumanEval ↑	0.1037	0.1098	**0.1220**	0.1037

this to non-determinism in training. It is however not due to a deficiency in our implementation, since our architecture, training configuration, and hyperparameters match exactly.

4.4 "Emergent Abilities" of Nanochat vs. Picochat

Below we present 10 questions that our nanochat (d24) model can answer correctly but our picochat (d16) model cannot. For each question, we show both models' responses and highlight where the picochat model fails.

1. **What's the capital city of Vietnam?**

d24: The capital of Vietnam is Hanoi. Hanoi is a city located in the central part of the country, with a population of over 9 million people. It is the largest city in Vietnam and is known for its rich history, cultural attractions, and modern amenities. Hanoi is also the political and economic center of the country, hosting various government institutions, international organizations, and major international organizations such as the United Nations.

d16: The capital of Vietnam is Ho Chi Minh City. It is located in the central part of the country and is a major city in the south-central region of Vietnam. (the capital is Hanoi)

2. **What is the element with atomic number 69?**

d24: The element with atomic number of 69 is Thulium. This element has 69 protons in its atomic nucleus.

d16: The element with atomic number 69 is uv = 69. This element has one atomic number, which is the number of protons in the atomic nucleus, and two atomic number, which is the number of neutrons in the nucleus. (produces incoherent output)

3. **What is 67×420?**

d24: To find the product 67×420, we need to multiply the two numbers together. This is a simple multiplication operation. $67 \times 420 = 28{,}140$. So, the product of 67 and 420 is 28,140.

d16: To find the value of 67×420, we can use the formula for the sum of a geometric series... Further simplifying, we get $S = 18{,}250$. Therefore, the value of 67×420 is 18,250. (applies an irrelevant formula and arrives at the wrong answer)

4. **What is the derivative of $x^2 + 3x$?**

d24: Applying the power rule to each term, we get the derivative of x^2 as $2x$, and the derivative of $3x$ as 3. Combining these results, the derivative of $x^2 + 3x$ is $2x + 3$.

d16: Applying this rule to each term, we get the derivative of x^2 as $2x$ and the derivative of $3x$ as $3 \cdot 2x$. Therefore, the derivative of $x^2 + 3x$ is $2x + 3 \cdot 2x$. (misapplies the power rule to the linear term)

5. **What is the symbol for element potassium?**

d24: The chemical symbol of potassium is the letter K.

d16: The chemical symbol for potassium is KO. In most cases, potassium is represented by the symbol KO. (the correct symbol is K)

6. **What is the boiling point of water in Fahrenheit?**

d24: The boiling point of water in Fahrenheit is 212°F.

d16: The boiling point of water in Fahrenheit is 108°F (21°C). (the correct answer is 212)

7. **What is the square root of 144?**

d24: The square root of 144 is 12 because 12 multiplied by itself equals 144.

d16: ...The square root of 144 is 13 (rounded to the nearest whole number, since it's a multiple of 4). (the correct answer is 12, and the reasoning is incoherent)

8. **When did World War I begin?**

d24: World War I began on July 28, 1914, when Austria-Hungary declared war on Serbia. Over the following days, Germany, Russia, France, and the United Kingdom entered the conflict due to alliance obligations and mobilizations.

d16: World War I began on June 17, 1914, in Europe, specifically in the city of Paris, France. (wrong date and location)

9. **Who wrote the play *Chicago*?**

d24: The play *Chicago* was written by Maurine Dallas Watkins, a journalist and playwright whose 1926 stage play later inspired the famous Broadway musical.

d16: The play *Chicago* was written by Daniel Burnham, a British novelist and journalist. (Daniel Burnham was an architect, not the playwright)

10. **What is the opposite of the word "ephemeral"?**

d24: The opposite of the word "ephemeral" is "permanent." An ephemeral word or phrase implies something that is fleeting, meaning it is temporary and may not persist over time.

d16: The word "ephemeral" has a neutral or negative connotation, which is the opposite of what you're looking for... (fails to provide an antonym and instead rambles about connotation)

Our qualitative evaluation comparing the d=16 (picochat) and d=24 (nanochat) models shows evidence of emergent abilities [16]. Although both models share the same architecture and training procedure, the larger model consistently demonstrates qualitatively different behavior across a range of factual, mathematical, and reasoning questions as tested above.

First, the $d = 24$ model demonstrates substantially stronger factual knowledge. For example, it correctly identifies Hanoi as the capital of Vietnam and Thulium as the element with atomic number 69, while the $d = 16$ model produces incorrect or nonsensical answers.

Second, the larger model demonstrates improved mathematical reasoning. For basic arithmetic (e.g., computing 67×420 or differentiating $x^2 + 3x$), the $d = 24$ model applies correct operations and thus produces correct results. Conversely, the $d = 16$ model hallucinates reasoning steps. This indicates that the picochat model has weaker internal representations of mathematical operations.

Third, the $d = 24$ model demonstrates improved linguistic understanding. For instance, when testing on semantic understanding, it correctly identifies "permanent" as the antonym of "ephemeral," whereas the smaller model fails to interpret the question. Similar improvements are observed for chemistry, physics, and historical knowledge.

Importantly, these improvements are not incremental improvements but qualitative shifts in behavior (consistent with an "emergent ability"). The smaller model often outputs incoherent text or hallucinated explanations, whereas the larger model consistently demonstrates appropriate reasoning.

Overall, our results provide limited qualitative evidence consistent with the emergence hypothesis [16]. Increasing model depth from $d = 16$ to $d = 24$ yields new capabilities in factual and linguistic understanding, as well as mathematical reasoning. The observed pattern from our evaluation here therefore aligns with prior findings that scaling language models unlocks new behaviors not present in smaller systems.

References

[1] Ebtesam Almazrouei, Hamza Alobeidli, Abdulaziz Alshamsi, Alessandro Cappelli, Ruxandra Cojocaru, Mérouane Debbah, Étienne Goffinet, Daniel Hesslow, Julien Launay, Quentin Malartic, Daniele Mazzotta, Badreddine Noune, Baptiste Pannier, and Guilherme Penedo. The falcon series of open language models, 2023.

[2] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayana Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. Palm: Scaling language modeling with pathways, 2022.

[3] Yann N. Dauphin, Angela Fan, Michael Auli, and David Grangier. Language modeling with gated convolutional networks, 2017.

[4] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In Jill Burstein, Christy Doran, and Thamar Solorio, editors, *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics.

[5] Gregory Kamradt. Needle in a haystack – pressure testing llms. https://github.com/gkamradt/LLMTest_NeedleInAHaystack, 2023. GitHub.

[6] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

[7] Andrej Karpathy. NanoGPT. https://github.com/karpathy/nanoGPT, 2022.

[8] Andrej Karpathy. Beating GPT-2 for «$100: the nanochat journey. https://github.com/karpathy/nanochat/discussions/481, 2026.

[9] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.

[10] Michael McCloskey and Neal J. Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. volume 24 of *Psychology of Learning and Motivation*, pages 109–165. Academic Press, 1989.

[11] Amirkeivan Mohtashami and Martin Jaggi. Random-access infinite context length for transformers. In *Proceedings of the 37th International Conference on Neural Information Processing Systems*, NIPS '23, Red Hook, NY, USA, 2023. Curran Associates Inc.

[12] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf, 2019. OpenAI Technical Report.

[13] Angela Sha. Csc490 nanochat tutorial. https://github.com/UofT-CSC490-W2026/022326-tutorial-nanochat, 2026. GitHub.

[14] Noam Shazeer. Glu variants improve transformer, 2020.

[15] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation language models, 2023.

[16] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, et al. Emergent abilities of large language models. *Transactions on Machine Learning Research*.

[17] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, Haoran Wei, Huan Lin, Jialong Tang, Jialin Wang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Ma, Jianxin Yang, Jin Xu, Jingren Zhou, Jinze Bai, Jinzheng He, Junyang Lin, Kai Dang, Keming Lu, Keqin Chen, Kexin Yang, Mei Li, Mingfeng Xue, Na Ni, Pei Zhang, Peng Wang, Ru Peng, Rui Men, Ruize Gao, Runji Lin, Shijie Wang, Shuai Bai, Sinan Tan, Tianhang Zhu, Tianhao Li, Tianyu Liu, Wenbin Ge, Xiaodong Deng, Xiaohuan Zhou, Xingzhang Ren, Xinyu Zhang, Xipin Wei, Xuancheng Ren, Xuejing Liu, Yang Fan, Yang Yao, Yichang Zhang, Yu Wan, Yunfei Chu, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, Zhifang Guo, and Zhihao Fan. Qwen2 technical report, 2024.

[18] Tianzhu Ye, Li Dong, Yuqing Xia, Yutao Sun, Yi Zhu, Gao Huang, and Furu Wei. Differential transformer, 2025.

[19] Biao Zhang and Rico Sennrich. Root mean square layer normalization. *Advances in neural information processing systems*, 32, 2019.